PRUEBA TÉCNICA · DESARROLLADOR

Programa de lealtad para tienda en línea

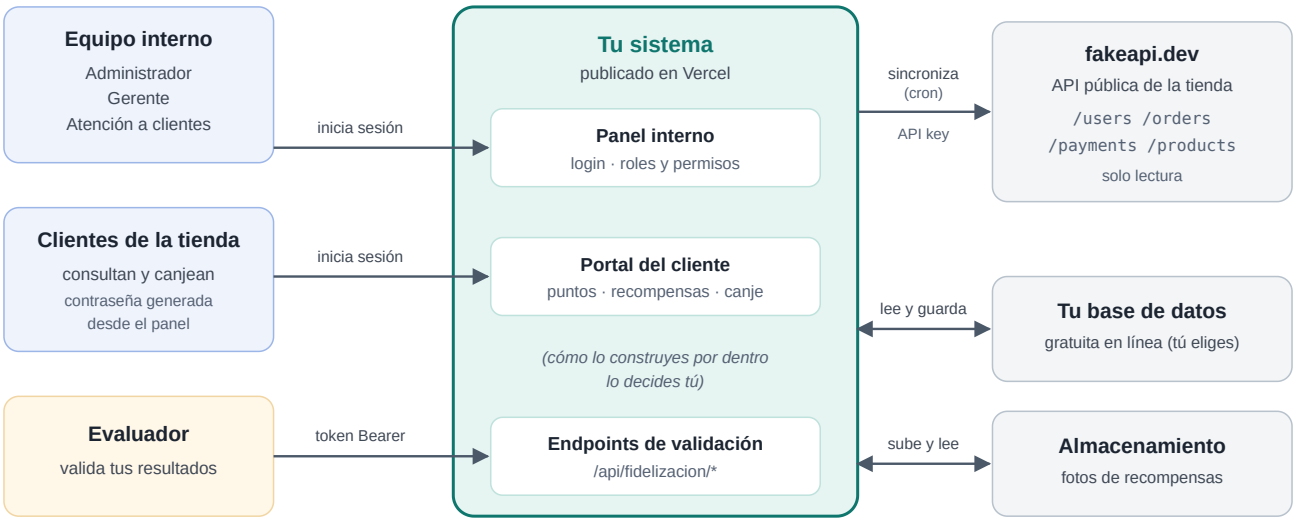
Instrucciones para el candidato

1. Contexto

Una tienda en línea quiere lanzar un **programa de lealtad**: sus clientes acumulan puntos por cada compra, suben de nivel según lo que gastan, **canjean sus puntos por productos o descuentos** desde un portal propio, y el equipo da seguimiento a quienes dejaron de comprar. El sistema lo usan dos tipos de personas: el **equipo interno** (con roles y permisos) y los **clientes** (en su portal). La tienda expone sus datos en la API **fakeapi.dev**: **clientes** (`/users`), **pedidos** (`/orders`) y **pagos** (`/payments`). Tu trabajo es construir el sistema que consuma esa API, guarde los datos en tu propia base de datos, calcule los puntos y el nivel de cada cliente y quede publicado en producción.

Fuente de datos	`https://fakeapi.dev/api/v1` · documentación en `https://fakeapi.dev/docs`
API key	Genera la tuya gratis en `https://fakeapi.dev/register` (solo nombre y correo, sin cuenta). Sube el límite de 100 a 1,000 requests por hora. Solo se muestra una vez: guárdala.
Fecha de entrega	**Sábado 24 de octubre de 2026, a más tardar a las 2:00 p.m.** (hora del centro de México). Entregas posteriores no se revisan.
Después	Llamada de **30 minutos**: explicas tu código y hacemos un cambio pequeño en vivo.
Tecnología	Libre (Next.js, Nuxt, SvelteKit, Express, etc.), siempre que se despliegue en **Vercel**.
Costo	**$0.** Todo debe funcionar en planes gratuitos: **Vercel Hobby**, base de datos y almacenamiento de fotos en su capa gratuita. No se aceptan servicios de pago ni pruebas gratuitas que expiren.

Diagrama de contexto



Las flechas indican quién inicia la comunicación. Una tarea programada actualiza tu base de datos desde fakeapi.dev; el panel y el portal leen de tu base de datos.

2. Qué tienes que entregar

1. **URL en producción en Vercel**, funcionando.
2. **Repositorio en GitHub** (público, o privado con acceso para nosotros) con historial de commits real.
3. **Base de datos en línea en plan gratuito** para guardar los datos sincronizados y los puntos: Supabase, Neon, Turso, MongoDB Atlas u otra. Tú eliges.
4. **README** con: cómo correrlo local, variables de entorno, las decisiones que tomaste (y por qué) y **dos diagramas tuyos**: el **modelo de datos** (tablas y relaciones) y la **arquitectura** (componentes, servicios y cómo se comunican). Pueden ser imágenes o Mermaid.

Responde al correo donde recibiste estas instrucciones con: la URL, el repositorio, **un usuario de prueba por cada rol** y **un cliente con su cuenta del portal ya activada** (correo y contraseña) y **un token de evaluación** para tus endpoints (sección 6). Las credenciales van solo en el correo, nunca en el repositorio.

Funcionalidad obligatoria

- **Acceso y seguridad:** login, usuarios, roles y permisos según la sección 3. Nada del sistema es accesible sin iniciar sesión.
- **Sincronización automática:** una **tarea programada (cron)** que descargue *todos* los clientes, pedidos y pagos de fakeapi.dev, actualice tu base de datos y recalcule los puntos **al menos una vez al día**, más un botón para sincronizar manualmente (solo con permiso). El panel y el portal leen siempre de tu base de datos.
  - Usa **Vercel Cron** con una ejecución diaria, que es lo que permite el plan gratuito.
  - Ese endpoint debe estar protegido con un secreto (por ejemplo `CRON_SECRET`): nadie más debe poder dispararlo.
  - Debe terminar dentro del tiempo máximo de ejecución de las funciones de Vercel y registrar cada corrida (inicio, fin, requests, resultado).

- **Cálculo de fidelización** según las reglas de la sección 4, guardando un **historial de movimientos de puntos** (no solo el saldo).
- **Panel web** (cada pantalla y acción según los permisos del usuario) con: resumen (clientes por nivel y top 10 por puntos), búsqueda de cliente con su detalle (puntos, nivel, historial), lista de clientes en riesgo y lista de cumpleaños próximos.
- **Portal del cliente y recompensas:** catálogo de productos y descuentos, canje de puntos y seguimiento de canjes, según la sección 5.
- **Endpoints JSON propios** con el formato exacto de la sección 6. Los usaremos para validar tus resultados automáticamente.

Puntos extra (opcionales)

- Sincronización incremental: que no vuelva a descargar ni recalcular todo si los datos no cambiaron.
- Pruebas automatizadas de las reglas de puntos.
- Mensaje sugerido (correo o WhatsApp) con un cupón para clientes en riesgo o de cumpleaños.
- Bitácora de auditoría: quién inició sesión, quién sincronizó, quién creó o modificó usuarios y roles.
- Bloqueo temporal tras varios intentos fallidos de inicio de sesión (equipo y clientes).
- Reporte de los datos inconsistentes que encontraste en la fuente.
- Importar productos de `/products` como base del catálogo de recompensas (después se editan nombre y foto).
- Notificaciones dentro del portal: canje registrado, producto entregado y cupón por vencer.

API key secreta. Usa tu key solo desde tu backend o funciones serverless y guárdala en las variables de entorno de Vercel. Si aparece en el navegador o en el repositorio, la prueba queda descalificada.

Límite de requests. fakeapi.dev permite 1,000 requests por hora con key. Tu sincronización debe paginar con `per_page=100`, manejar errores y respuestas `429`, y no consultar la fuente en cada visita al panel.

Datos reales, no ideales. La fuente no siempre es consistente: hay tipos de dato mezclados, varias monedas y referencias que no cuadran. Valida y normaliza lo que recibes.

3. Usuarios, roles y permisos

Es lo primero del sistema. Requisitos mínimos:

- **Login** con correo y contraseña; contraseñas guardadas con hash (bcrypt, argon2 o el de tu framework); sesión que expira; cierre de sesión.
- **Usuarios:** el administrador crea, edita y desactiva usuarios y les asigna un rol. No se puede desactivar al último administrador.
- **Roles editables:** el administrador puede crear roles nuevos y marcar qué permisos tiene cada uno. Los permisos son una lista fija definida en tu código.
- **Permisos validados en el backend:** cada endpoint verifica el permiso y responde `403` si no lo tiene. Ocultar botones en pantalla no basta.

- **Clientes aparte:** los clientes del portal son un tipo de cuenta distinto: no tienen roles internos y solo pueden ver y canjear lo suyo.
- **Tokens de API:** el administrador genera y revoca tokens para consumir tus endpoints de validación (sección 6). Guarda solo su hash.

Roles iniciales que deben existir (con estos permisos por defecto):

PERMISO	ADMINISTRADOR	GERENTE	ATENCIÓN A CLIENTES
Ver resumen del panel	✓	✓	✓
Buscar cliente y ver puntos, nivel e historial	✓	✓	✓
Ver montos (gasto en MXN, pedidos y pagos)	✓	✓	—
Ver listas de riesgo y cumpleaños	✓	✓	✓
Exportar reportes (CSV)	✓	✓	—
Gestionar catálogo de recompensas	✓	✓	—
Gestionar canjes (entregar, aplicar cupón, cancelar)	✓	✓	✓
Generar o restablecer acceso al portal de un cliente	✓	✓	✓
Ejecutar sincronización	✓	—	—
Gestionar usuarios	✓	—	—
Gestionar roles y permisos	✓	—	—
Gestionar tokens de API	✓	—	—

4. Reglas de negocio

Todas las reglas se evalúan a la **fecha de corte: 30 de abril de 2026** (`2026-04-30`). **No uses la fecha de hoy.** Las fechas de la API vienen en UTC; conviértelas a **hora de México** (UTC−6 fijo, sin horario de verano) y usa la fecha local resultante. Ignora pagos y pedidos con fecha local posterior al corte, y los de clientes que no existan en `/users`.

REGLA	DEFINICIÓN
Conversión a MXN	`monto × tipo de cambio`, redondeado a centavos. Tipos de cambio fijos: **USD 18.50 · EUR 20.00 · GBP 23.50 · CAD 13.50 · AUD 12.00**. Un pago en otra moneda se ignora.
Puntos por pago	1 punto por cada $10 MXN de cada pago con `status = succeeded`, redondeando hacia abajo por pago: `floor(mxn / 10)`. Un pago `refunded` genera sus puntos y después un movimiento negativo por la misma cantidad (neto 0). `failed` y `disputed` (o cualquier otro estado) no generan puntos. La fecha de un pago es `paid_at`, o `created_at` si `paid_at` es null.
Bono por pedido	+250 puntos por cada pedido (orden) con `status = delivered` **y** `payment_status = paid`. Fecha: `created_at` del pedido.
Gasto 12 meses y nivel	Suma en MXN de pagos `succeeded` con fecha mayor a `corte − 365 días` y hasta el corte inclusive. **Bronce:** menos de $150,000 · **Plata:** de $150,000 a $400,000 · **Oro:** más de $400,000.
Cliente en riesgo	Tiene al menos un pago `succeeded` **y** lleva 90 días o más sin actividad al corte. **Actividad** = fecha de un pago `succeeded` o de un pedido que no esté `cancelled`. `dias_sin_actividad` = días entre la última actividad y el corte.
Cumpleaños próximos	Clientes cuyo `dob` (tómalo como fecha de calendario: sus primeros 10 caracteres, sin convertir zona horaria) cae, en mes y día, entre el corte y el corte + 7 días, ambos inclusive.

5. Portal del cliente y recompensas

Portal del cliente

- **Cuentas:** la sincronización crea una cuenta de portal por cada cliente de `/users`, en estado **sin acceso**. Los correos de fakeapi.dev son ficticios, así que **no se envían correos**.
- **Acceso generado desde el panel:** en la ficha del cliente, un usuario interno con permiso da clic en **Generar acceso**; el sistema crea una **contraseña temporal aleatoria**, la muestra **una sola vez** y guarda solo su hash. **Restablecer acceso** genera otra e invalida la anterior.
- **Primer ingreso:** el cliente entra con su correo y la contraseña temporal, y el sistema lo obliga a crear una nueva antes de continuar. Prohibido usar contraseñas fijas o predecibles (como la fecha de nacimiento).
- **Qué ve:** puntos disponibles, nivel y cuánto le falta para el siguiente, historial de movimientos, catálogo de recompensas, sus canjes y sus cupones.
- **Aislamiento:** un cliente nunca puede ver ni canjear con los datos de otro, aunque cambie IDs en la URL o en las peticiones.

Catálogo de recompensas (lo administra el equipo interno)

CAMPO	PRODUCTO	DESCUENTO
Comunes	Foto, nombre, descripción, **puntos requeridos**, **stock** (para descuentos puede ser ilimitado), **nivel mínimo** (Bronce, Plata u Oro), **límite de canjes por cliente** (opcional) y activo / inactivo.	
Propios	—	Tipo (**porcentaje** o **monto fijo en MXN**), valor, compra mínima (opcional) y **vigencia en días** a partir del canje.
Al canjear se genera	Un canje **pendiente de entrega**.	Un **cupón con código único** (ej. `LEAL-7Q4K-92XM`) y fecha de vencimiento.
Estados	`pendiente` → `entregado` o `cancelado`	`vigente` → `usado`, `vencido` o `cancelado`

Las fotos se suben a un almacenamiento en plan gratuito (Supabase Storage, Vercel Blob, Cloudinary u otro), nunca dentro de la base de datos.

Reglas del canje

REGLA	DEFINICIÓN
Puntos disponibles	`puntos ganados − puntos canjeados`. Los puntos ganados son los de la sección 4. Un canje cancelado devuelve sus puntos; un cupón vencido no.
Validaciones	El canje procede solo si la recompensa está activa, hay stock, el nivel del cliente es igual o mayor al nivel mínimo, no excede su límite por cliente y tiene puntos disponibles suficientes. Si algo falla, el cliente ve un mensaje claro de por qué.
Todo o nada	El canje descuenta el stock, registra un movimiento `canje` negativo en el historial y crea el canje o cupón **en una sola transacción**. Si dos clientes canjean la última unidad al mismo tiempo, solo uno debe lograrlo, y el saldo nunca puede quedar negativo.
Cancelación	Al cancelar, se devuelve el stock y se registra un movimiento `devolucion` positivo por los mismos puntos.
Sincronización	Volver a sincronizar con fakeapi.dev no debe borrar, duplicar ni alterar los canjes.

6. Tus endpoints de validación (formato obligatorio)

Publícalos en tu app de Vercel **protegidos con token**: deben exigir `Authorization: Bearer <token>` con un token generado en tu panel, y responder `401` sin token o con uno inválido. Los consultamos desde un navegador, así que deben habilitar CORS: `Access-Control-Allow-Origin: *`, permitir el header `Authorization` y responder la petición `OPTIONS`. Usa estas rutas y campos exactos; puedes agregar campos extra.

```
GET /api/fidelizacion/clientes/{id}
{
  "cliente_id": 241,
  "puntos": 13490,                  // ganados según la sección 4 (sin restar canjes)
  "puntos_disponibles": 11990,       // ganados − canjeados
  "nivel": "Plata",                 // "Bronce" | "Plata" | "Oro"
  "gasto_12m": 184220.35,            // MXN
  "en_riesgo": false,
  "dias_sin_actividad": 12,          // null si nunca tuvo actividad
  "movimientos": [
    { "fecha": "2026-03-10", "tipo": "pago",
      "referencia": "pago:3220", "puntos": 5547 },
    { "fecha": "2026-03-10", "tipo": "reembolso",
      "referencia": "pago:3220", "puntos": -5547 },
    { "fecha": "2026-04-02", "tipo": "bono_orden_entregada",
      "referencia": "orden:3047", "puntos": 250 },
    { "fecha": "2026-10-22", "tipo": "canje",
      "referencia": "canje:18", "puntos": -1500 }
  ]
}

GET /api/fidelizacion/riesgo
[ { "cliente_id": 192, "dias_sin_actividad": 140 }, ... ]

GET /api/fidelizacion/cumpleanos
[ { "cliente_id": 77, "fecha_cumpleanos": "2026-05-03" }, ... ]

// las dos listas deben venir completas, sin paginar
```

7. La API fuente (fakeapi.dev)

URL base	`https://fakeapi.dev/api/v1`
Autenticación	Header `Authorization: Bearer <tu-key>` (recomendado) o parámetro `?api_key=<tu-key>`.
Paginación	`page` y `per_page` (máximo 100). Respuesta: `{ "data": [...], "meta": { "total", "page", "per_page", "total_pages", "has_more" } }`
Límite	1,000 requests por hora con key (100 sin key). Headers `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`.

Recursos que necesitas

ENDPOINT	CAMPOS RELEVANTES
`GET /users`	`id, name, email, phone, dob, created_at`
`GET /orders`	`id, status` (pending, processing, shipped, delivered, cancelled), `payment_status` (paid, pending, failed), `total, user.id, created_at`
`GET /payments`	`id, amount, currency, status` (succeeded, failed, refunded, disputed), `paid_at, created_at, user.id, metadata.order_id`

Consulta la documentación oficial para el detalle de cada campo. Ten en cuenta que la fuente es de uso público: diseña tu sincronización para que sea idempotente (correrla dos veces no debe duplicar puntos).

8. Cómo te evaluamos

ASPECTO	QUÉ REVISAMOS
Seguridad y acceso	Login, contraseñas con hash, roles editables, permisos validados en el backend (probamos acciones prohibidas con cada rol), endpoints que rechazan peticiones sin token.
Portal y canje	Acceso generado desde el panel y cambio obligatorio en el primer ingreso, aislamiento entre clientes, validaciones del canje, transacción todo o nada (probamos dos canjes simultáneos de la última unidad) y estados de canjes y cupones.
Exactitud	Puntos, nivel, gasto, riesgo y cumpleaños contra nuestra hoja de respuestas (validación automática con tus endpoints).
Consumo de la API	Paginación completa, manejo de errores y del límite de requests, sincronización idempotente.
Modelo de datos	Diseño de las tablas y del historial de puntos.
Código	Organización, claridad, manejo de errores y de secretos.
Producción	Que funcione en Vercel y que la API key nunca quede expuesta.
Defensa	Tu explicación en la llamada y el cambio en vivo.

Puedes usar IA como herramienta, pero debes entender y poder defender cada línea que entregues. ¡Éxito!